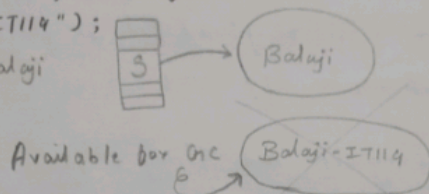


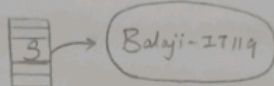
java.lang.String

CASE: 1

```
String s = new String("Balaji");  
s.concat("-IT114");  
System.out.println(s);
```



```
StringBuffer sb = new StringBuffer("Balaji");  
sb.append("-IT114");  
System.out.println(sb);
```

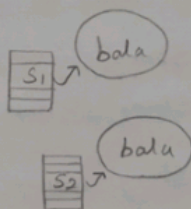


→ Once we create a String object, we can't perform any change in the existing object. If we are trying to perform any change, then with those changes a new object will be created. This non-changeable behaviour is nothing but Immutability of String.

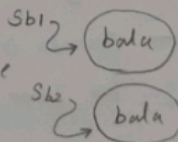
→ Once we create StringBuffer object, we can perform any change with the existing object. This changeable behaviour is called Mutability of StringBuffer.

CASE: 2

```
String s1 = new String("bala");  
String s2 = new String("bala");  
System.out.println(s1 == s2); // false  
System.out.println(s1.equals(s2)); // true
```



```
StringBuffer sb1 = new StringBuffer("bala");  
StringBuffer sb2 = new StringBuffer("bala");  
System.out.println(sb1 == sb2); // false  
System.out.println(sb1.equals(sb2)); // true
```



→ In String class, equals() method is overridden for content comparison. Hence, even though objects are different, the content is same, so equals() method returns true.

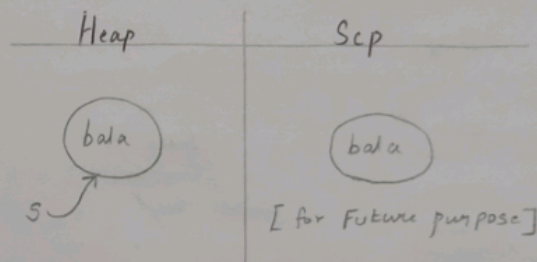
Starting from Java-7, the SCP was moved from Method area to the Heap Area.

→ In StringBuffer class, equals() method is not overridden for content comparison. Hence, if objects are different, its parent class equals() method returns false as it performs Reference comparison.

CASE: 3

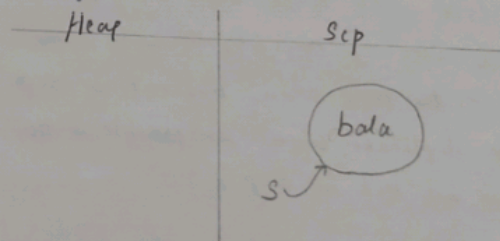
```
String s = new String("bala");
```

→ In this case, two objects will be created: one in the heap area and the other in the SCP [String Constant Pool] and 's' is always pointing to the heap object.



```
String s = "bala";
```

→ In this case, only one object will be created in the SCP and 's' always points to that object.



→ Object creation in the SCP is always optional. First, it will check if there is any object already present in the SCP with the required content. If yes, then the existing object will be reused. If not already available, then only a new object will be created.

→ But this rule is applicable only for the SCP and not for the heap.

→ One is not allowed to access the SCP area. Hence, even though an object doesn't contain a reference variable, it is not eligible for GC if it is present in the SCP area.

→ One concept is for the heap memory area, whereas the SCP is present in the method area.

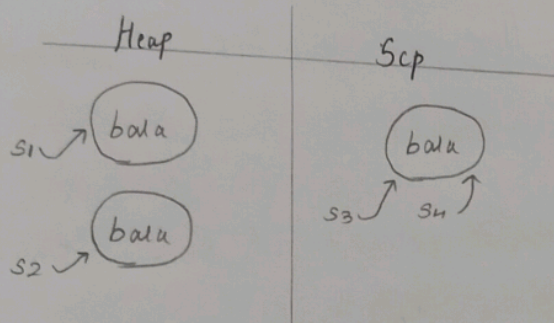
→ All GC Objects will be destroyed automatically at the time of JVM shutdown.
 If it is a Web application, Server shutdown.
 But it is never recommended to shutdown the Server, but we can go for scheduled Restart. If it is a clustered Environment, the other Servers provide support even one server is in Restarting process.

```
String s1 = new String("bala");
String s2 = new String("bala");
```

```
String s3 = "bala";
```

```
String s4 = "bala";
```

has



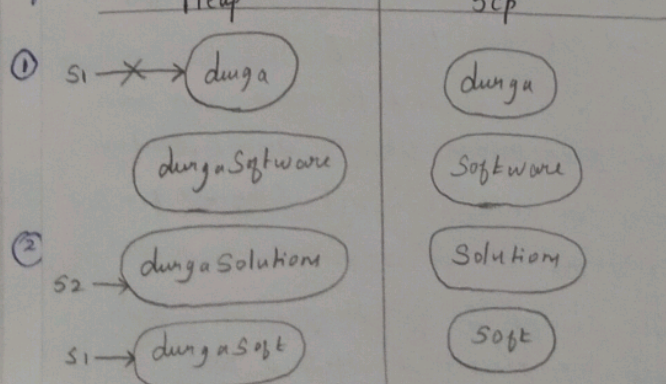
C

→ Whenever we are using new operator, Compulsory a new Object will be created in the heap area. Hence, there may be a chance of Existing two Object with same Content in the heap area, but not in scp. (That is duplicate Object are possible in the heap area but not in scp)

```
String s1 = new String("durga");
s1.concat("Software");
String s2 = s1.concat("Solutions");
s1 = s1.concat("Soft");
System.out.println(s1); // durgasoft
System.out.println(s2); // durgasolutions
```

W

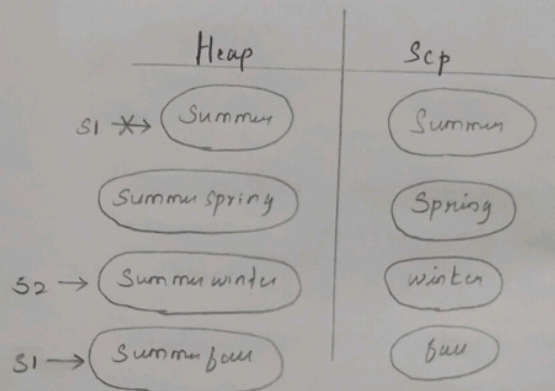
ay



→ For Every String Constant, One Object will be placed in scp area.

→ Because of some runtime operations if an Object is required to create, that Object will be placed Only in the heap area but not in scp area.

```
String s1 = new String("Summer");
s1.concat("Spring");
String s2 = s1.concat("Winter");
s1 = s1.concat("Fall");
System.out.println(s1); // SummerFall
System.out.println(s2); // SummerWinter
```



Constructors :

① String s = new String();
 → creates an Empty String Object

String s = "";
 String s = null;
 String Object is there but 0-length.
 String Object itself not there.

② String s = new String(String literal);
 → creates a String Object on the heap for the given String literal

③ String s = new String(StringBuffer sb);
 → creates an Equivalent String Object for the given StringBuffer.

④ String s = new String (char[] ch);
→ Create an Equivalent String Object for the Given char[]

```
char[] ch = { 'a', 'b', 'c', 'd' };  
String s = new String (ch);  
System.out.println(s); // abcd ✓
```

⑤ String s = new String (byte[] b);
→ Create an Equivalent String Object for the Given byte[]

```
byte[] b = { 100, 101, 102, 103 };  
String s = new String (b);  
System.out.println(s); // defg ✓
```

⋮

Important Methods:

① public char charAt (int index) { ... }

```
String s = "balaji";  
System.out.println(s.charAt(0)); // b  
System.out.println(s.charAt(30));
```

PE: StringIndexOutOfBoundsException

→ Return the Character locating at Specified index.

② public String concat (String s) { ... }

→ The Overloaded + and += operators also meant for Concatenation purpose only.

```
String s = "bala";  
s = s.concat("Software");  
// s = s + "Software";  
// s += "Software";  
System.out.println(s); // balaSoftware
```

③ public boolean equals (Object o)

→ To perform Content Comparison, where Case is important.

→ This is Overriding Version of Object class equals() Method.

④ public boolean equalsIgnoreCase (String s)

→ To perform Content Comparison where, Case is not important.

```
String s = "java";  
System.out.println(s.equals("JAVA"));  
System.out.println(s.equalsIgnoreCase("JAVA"));
```

Output: false
true

→ In General we can use equals() Method to validate username, Email ID where Case is not important. whereas we can use equalsIgnoreCase() Method to validate password where Case is important.

⑤ public String substring (int begin);

→ Return substring from begin index to End of the String

⑥ public String substring (int b, int e);

→ Return substring from begin index to End-1 index

```
String s = "abcdefg";  
System.out.println(s.substring(3));  
System.out.println(s.substring(0, 6));
```

Output: defg
cdef

⑦ public int length()

→ Return number of character present in the String

```
String s = "bala";  
System.out.println(s.length()); X
```

CE: Cannot find length

Symbol: Variable length

Location: java.lang.String

```
System.out.println(s.length()); // 7
```

→ length var applicable for arrays but not for String object. whereas length() Method applicable for String Object but not for arrays.

→ ⑧ public String replace (char o, char n)
 boolean → Replace the old character with new
 character in the given String
 Example
 String s = "ababa";
 System.out.println(s.replace('a', 'b')); // bbbba
 ⑨ public String toLowerCase();
 public String toUpperCase();
 ⑩ public String trim();
 → To remove the blank space present at
 beginning and end of the string, but not
 middle blank spaces.

⑪ public int indexOf (char ch);

→ Returns the index of first occurrence of
 specified character.

⑫ public int lastIndexOf (char ch);

→ Returns the index of last occurrence of
 specified character.

String s = "ababa";

System.out.println(s.indexOf('a')); // 0

System.out.println(s.lastIndexOf('a')); // 4

(*****)

→ Because of runtime operation and if there
 is a change in the content, then with
 those changes a new object will be
 created on the heap.

→ If there is no change in the content,
 then existing object will be reused and
 the new object won't be created, unless
 created with 'new' operator.

→ Whether the object present in heap
 or sep, the rule is same.

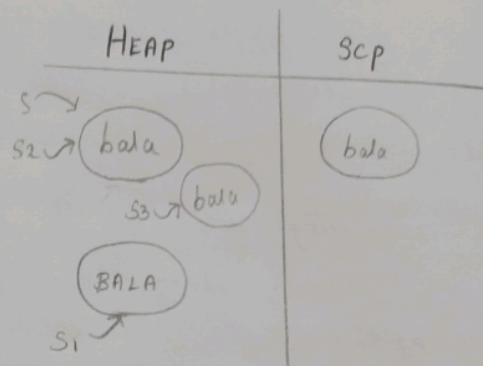
String s = new String ("bala");

String s1 = s.toUpperCase();

String s2 = s.toLowerCase();

// String s3 = new String (s.toLowerCase());

System.out.println (s == s1); // false
 System.out.println (s == s2); // true
 System.out.println (s == s3); // false



*** One more twist

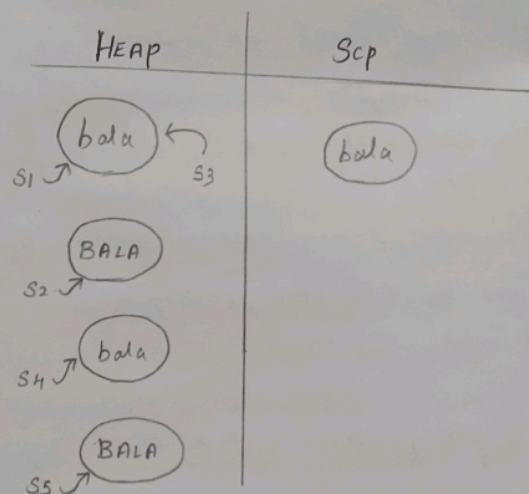
String s1 = new String ("bala");

String s2 = s1.toUpperCase();

String s3 = s1.toLowerCase();

String s4 = s2.toLowerCase(); *

String s5 = s4.toUpperCase(); *



String s1 = "bala";

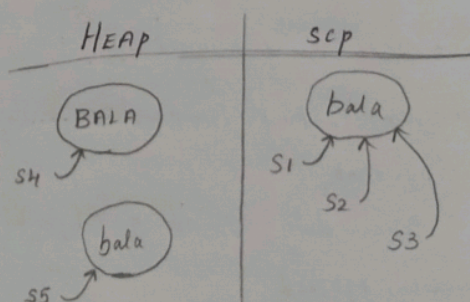
String s2 = s1.toUpperCase();

System.out.println (s1 == s2); // true

String s3 = s1.toLowerCase();

String s4 = s1.toUpperCase();

String s5 = s4.toLowerCase();



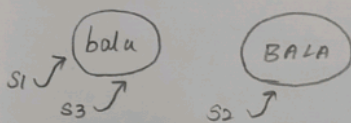
How to Create Our Own Immutable class?

→ Once we create an Object, we can't perform any changes in that Object. If we are trying to perform any change and because of that if there is a change in the content, then with those changes a new Object will be created. If there is no change in the content, then the existing Object will be reused. This behaviour is called Immutability.

```
String s1 = new String("bala");
```

```
String s2 = s1.toUpperCase();
```

```
String s3 = s1.toLowerCase();
```



→ We can create our own immutable class, if we bring the same behaviour as above.

```
final public class Tut {
```

```
    private int i;
```

```
    Tut(int i) {
```

```
        this.i = i;
```

```
    }
```

```
    public Tut modify(int i) {
```

```
        if (this.i == i) {
```

```
            return this;
```

```
        }
```

```
        else {
```

```
            return new Tut(i);
```

```
        }
```

```
    }
```

```
}
```

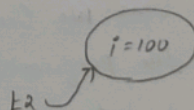
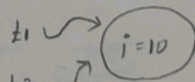
```
Tut t1 = new Tut(10);
```

```
Tut t2 = new Tut(100); t1.modify();
```

```
Tut t3 = new Tut(10); t1.modify();
```

```
System.out.println(t1 == t2); // false
```

```
System.out.println(t1 == t3); // true
```



→ We are getting immutable nature, because of the `modify()` method, so no one is allowed to change its implementation. Compulsory we have to declare every immutable class as final.

→ As per the previous example, once we create Test Object, we can't perform any change on the Test Object. If we are trying to perform any change and because of that if there is a change in the content, then with those changes a new Object will be created. If there is no change in the content then existing Object will be reused.

final Vs Immutability

→ final applicable for variables, but not for Object. Whereas immutability applicable for Object but not for Variables.

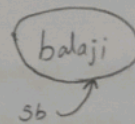
→ By declaring a Ref-Var as final we won't get any immutability nature. Even though ref-var is final, we can perform any type of change in the corresponding Object. But we can't perform reassignment for that Variable. Hence, final and immutable, both are different concepts.

```
final StringBuffer sb = new StringBuffer("bala");
```

```
sb.append("ji");
```

```
System.out.println(sb); // balaji
```

```
X sb = new StringBuffer("Hai");
```



CE: Cannot assign a value to a final Variable sb

Which of the following is meaningful?

- ① final Variable ✓
- ② final Object ✗
- ③ immutable Variable ✗
- ④ immutable Object ✓

BALA

51

rent at

**** One more twist

but

Final public class Tut {

private int i;

Tut(int i) { this.i = i; }

public Tut modify(int i) {

if (this.i == i) {

return this;

}

else {

return new Tut(i);

}

}

}

public class Tut {

psvm (String[] args) {

Tut t1 = new Tut(10);

Tut t2 = t1.modify(100);

Tut t3 = t1.modify(10);

System.out.println(t1.hashCode());

System.out.println(t2.hashCode());

System.out.println(t3.hashCode());

System.out.println(t1 == t2);

System.out.println(t1 == t3);

}

}

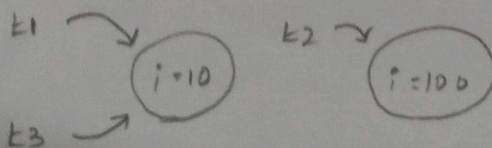
Output: 798154996

681842940

798154996

false

true



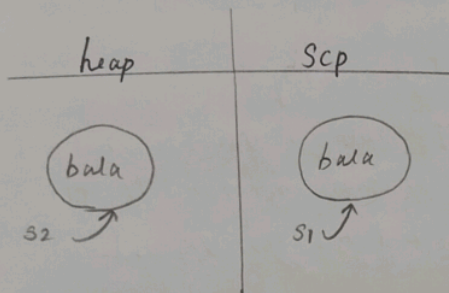

```
String s1 = "";
String s2 = new String();
```

→ In both cases object will be created but with no content. First one in Scp and the next one in Heap

```
System.out.println(s1 == s2); // false
System.out.println(s1.equals(s2)); // true
```

```
String s1 = "bala";
String s2 = new String("bala");
System.out.println(s1.hashCode()); // 97950431
System.out.println(s2.hashCode()); // 97950431
System.out.println(s1 == s2); // false
System.out.println(s1.equals(s2)); // true
```

Eg:1



→ Even though both reference's hashCode() are equals, == operator returns false this is because both references are not pointing to the same object.

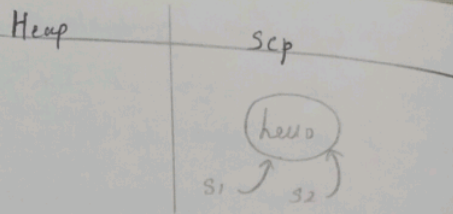
→ Same hashCode(), means that the hashCode is computed based on the string's content by using a formula. *****

→ for "bala" → 97950431 is the hashCode always by that formula.

“ If two objects have same hashCode, it doesn't mean that their references are same especially in case of String objects” [Refer Eg:1, Eg:2, Eg:3]

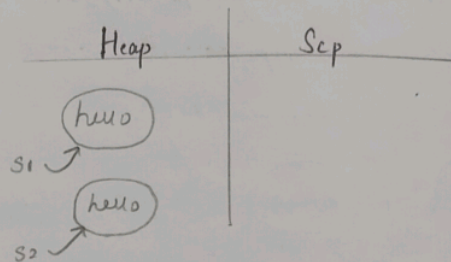
```
String s1 = "hello";
String s2 = "hello";
System.out.println(s1.hashCode()); // 97950431
System.out.println(s2.hashCode()); // 97950431
System.out.println(s1 == s2); // true
System.out.println(s1.equals(s2)); // true
```

Eg:2



```
String s1 = new String("hello");
String s2 = new String("hello");
System.out.println(s1.hashCode()); // 97950431
System.out.println(s2.hashCode()); // 97950431
System.out.println(s1 == s2); // false
System.out.println(s1.equals(s2)); // true
```

Eg:3



StringBuffer:

→ If the content is fixed and won't change frequently then it is recommended to go for string.

→ If the content is not fixed and keep on changing then it is not recommended to use string, because for every change a new object will be created, which affects performance of the system.

→ To handle this requirement we should go for StringBuffer.

→ The main advantage of StringBuffer over string is, all required changes will be performed in existing object only.

→ Once we created a string object. No change can be performed on it

“bala” → length(4)

→ Once we created a StringBuffer object we can perform any changes on the existing object itself. So, we have to talk about two terms

→ length()
→ capacity()

length() → how many characters are there in the StringBuffer Object?

Capacity() → how many characters possible to add in the StringBuffer Object?

Constructors:

① `StringBuffer sb = new StringBuffer();`

→ creates an Empty StringBuffer Object with default initial Capacity - 16. Once SB reaches its Max-Capacity a new SB Object will be created with

$$\text{new Capacity} = (cc + 1) * 2;$$

```
StringBuffer sb = new StringBuffer();
```

```
sopln (sb.capacity()); // 16
```

```
sb.append("abcdefghijklmnop");
```

```
sopln (sb.capacity()); // 16
```

```
sb.append("q");
```

```
sopln (sb.capacity()); // 34 (16+1)*2
```

② `StringBuffer sb = new SB (int initial capacity);`

→ creates an Empty StringBuffer Object with specified initial capacity. The biggest advantage of this constructor is the performance level going to improve.

③ `StringBuffer sb = new SB (String s);` ***

→ creates an Equivalent SB for the given

String with $\text{Capacity} = s.length + 16;$

```
StringBuffer sb = new StringBuffer("bala");
```

```
sopln (sb.capacity()); // 20 (4+16)
```

④ `StringBuffer sb = new SB (char sequence s);`

→ creates an Equivalent SB that contains the same characters as the specified char sequence

Important Methods:

① `public int length();`

② `public int capacity();`

③ `public char charAt (int index);`

```
StringBuffer sb = new SB("bala");
```

```
sopln (sb.charAt(0)); // b
```

```
sopln (sb.charAt(10));
```

RE: `StringIndexOutOfBoundsException`

There is no such Exception like

'StringBuffer Index OutOfBounds Exception'

④ `public void setCharAt (int i, char ch);`

→ To replace a character located at specified index with provided character.

⑤ `public StringBuffer append (String s)` ***

Overloaded Methods

- (int i)
- (long l)
- (char ch)
- (boolean b)
- ⋮

```
StringBuffer sb = new SB();
```

```
sb.append("PI Value is: ");
```

```
sb.append(3.14);
```

```
sb.append(" It is Exactly");
```

```
sb.append(true);
```

```
sopln (sb);
```

OUTPUT: `PI Value is: 3.14 It is Exactly true`

→ Add only at the End of the SB.

⑥ `public StringBuffer insert (int i, String s)`

Overloaded Methods

- (int i, int j)
- (int i, char ch)
- (int i, boolean b)
- ⋮

```
StringBuffer sb = new SB("abcdefgh");
```

```
sb.insert(2, "xyz");
```

```
sopln (sb); // abxyz cdefgh
```

⑦ `public SB delete (int s, int e)`

→ To delete characters from s index to e-1 index

⑧ public void deleteCharAt (int index)
 → To delete the character from the specified index

⑨ public String reverse();
 StringBuffer sb = new SB ("bala");
 Sopl (sb.reverse()); // alab

⑩ public void setLength (int length);
 StringBuffer sb = new SB ("balaji");
 sb.setLength(4);
 Sopl (sb); // bala (The remaining char will automatically removed)

⑪ public void ensureCapacity (int capacity);
 → To increase capacity on fly based on our requirement. That is I already have SB Object with default initial capacity, But now I want to Increase capacity

StringBuffer sb = new SB();
 Sopl (sb.capacity()); // 16
 sb.ensureCapacity(1000);
 Sopl (sb.capacity()); // 1000
 sb.ensureCapacity(15);
 Sopl (sb.capacity()); // 1000

⑫ public void trimToSize();
 → To deallocate Extra allocated free memory

StringBuffer sb = new SB(1000);
 sb.append("abc");
 sb.trimToSize();
 Sopl (sb.capacity()); // 3

⑬ public int compareTo (SB another)
 → Compare characters by their Unicode Value (Lexicographical Comparing)
 → Return < 0 → SB < another
 0 → SB == another
 > 0 → SB > another

StringBuffer sb1 = new SB("apple");
 StringBuffer sb2 = new SB("banana");
 Sopl (sb1.compareTo(sb2)); // -1
 → apple vs banana
 a → 97
 b → 98 } 97 - 98 = -1

NOTE: AbstractString Builder is an abstract class in the java.lang package that serves as the foundation of Java's mutable string classes

↳ String Builder
 ↳ String Buffer

→ It provides core functionality for manipulating character sequence like appending, inserting, deleting but it isn't directly used - We can't create object for this class, as it is ~~final~~ abstract.

66 AbstractSB is a Blueprint, which defines how to build and modify a string and its subclasses add the final touch (like thread safety).

→ It internally uses byte[] with a code field (LATINI or UTF16) for storage. Earlier versions used char[], but by using byte[], memory usage got halved.

used to switch between (LATINI & (ie; b/n byte[] & char[]) UTF16)

→ If we note down, Many of the methods of the ~~String Builder~~ StringBuffer class has a call to super, which is nothing but to AbstractString Builder.

StringBuffer methods have some common point in there.

→ *** Nearly all the methods present in the StringBuffer class are synchronized. That is StringBuffer Objects are thread-safe by default. Only one thread is allowed to access the SB object at a time. This leads to performance issue.

StringBuilder:

→ To handle the performance issue of the StringBuffer, SUN people introduced StringBuilder Concept in 1.5 Version

→ StringBuilder is exactly same as StringBuffer, except the following differences.

StringBuffer	StringBuilder
→ Extends ASB	→ Extends ASB
→ Every method in StringBuffer is Synchronized.	→ Every method in StringBuilder is non-synchronized.
→ At a time only one thread is allowed to operate on SB object and hence SB object is Thread-safe.	→ At a time multiple threads are allowed to operate on SB object & hence SB object are not Thread-safe
→ Threads are required to wait to operate on SB object, which makes the performance low.	→ Threads are not required to wait to operate on SB object, hence performance is high
→ Introduced in 1.0V	→ Introduced in 1.5V
→ Guarantee Data Consistency	→ Don't guarantee for Data consistency
→ Follows sequential Execution	→ Follows parallel Execution
→ More Execution time	→ Less Execution time

→ Except the above differences, Everything is same in StringBuffer and StringBuilder, including Methods and Constructors.

String Vs StringBuffer Vs StringBuilder

→ If the content is fixed and won't change frequently then we should go for String.

→ If the content is not fixed and keep on changing, But Thread Safety is required then, we should go for StringBuffer.

→ If the content is not fixed and keep on changing, But Thread Safety is not required, then we should go for StringBuild.

→ String is immutable, whereas StringBuffer & StringBuilder are mutable.

Method chaining:

→ For most of the methods in String, StringBuffer and StringBuilder, return type are same type. Hence, after applying a Method on the result, we can call another Method, which forms Method chaining.

Sb.m1().m2().m3()....

→ In Method chaining, method calls will be executed from left to right

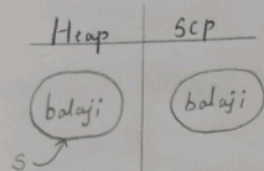
```
StringBuffer sb = new SB();  
sb.append("bala").append("ji").  
append("s").insert("XYZ").reverse();  
Sopln(sb); ✓ 100% para
```

→ Method chaining is the very common property in String, StringBuffer and StringBuilder.

RECAP:

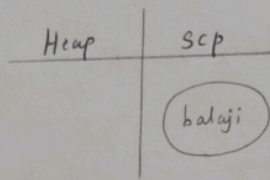
```
String s = new String("balaji");
```

2 Objects



```
String s = "balaji";
```

1 Object



SCP → String Constant pool

- If any Object got Created because of String Constant Literal, One Object is Created in SCP area with no Explicit reference unless provided. But implicit reference Var will be Maintained by JVM. Hence it is not Eligible for GC.

→ Until 1.6V SCP area is a part of Method area / PERMANENT. But from 1.7V

- (Onwards for Efficient memory Utilization, SCP area will be moved to Heap area.

- (→ SCP Object for reusability in the future purpose.

→ Whenever we are using 'new' operator, Compulsory a new Object will be Created in the Heap area and because of this there is a possibility for two Object with same content in Heap area, but no possibility in SCP.

```
String s1 = new String("You cannot change me");
```

```
String s2 = new String("You cannot change me");
```

```
Sopln(s1 == s2); // false
```

```
String s3 = "You cannot change me";
```

```
String (s1 == s3); // false
```

```
String s4 = "You cannot change me";
```

```
Sopln(s3 == s4); // true
```

```
String s5 = "You cannot" + "change me";
```

```
Sopln(s4 == s5); // true
```

As both are Constants, Compiler itself perform the Concatenation, At runtime s5 is same as s4. So, Object will be placed in SCP if not already there.

```
String s6 = "You cannot";
```

```
String s7 = s6 + "change me";
```

```
Sopln(s4 == s7); // false
```

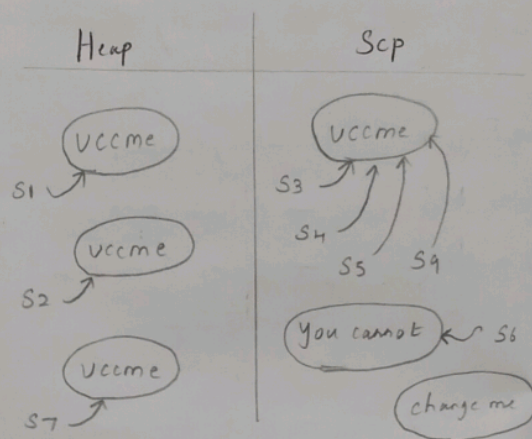
If atleast One Argument is Variable, then Concatenation happens at runtime & Object placed in heap.

```
final String s8 = "You cannot";
```

```
String s9 = s8 + "change me";
```

```
Sopln(s4 == s9); // true
```

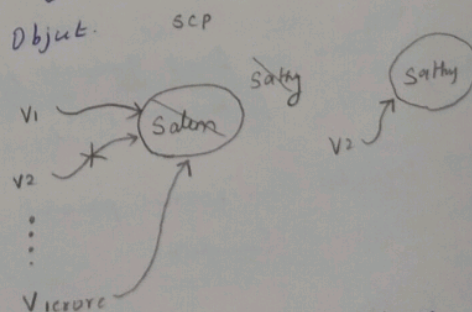
All final Variables are replaced with values during Compilation Itself.



Importance of SCP: [Example - Voter Reg form]

→ In SCP, Same Object can be referenced with multiple ref. vars. So, Instead of Creating Same Content Object Multiple times we can reuse the already Created Object. Memory Utilization is improved.

→ Here there is one biggest disadvantage. Consider a Object in scp with city name 'Salem'. One more references are there for this Object.



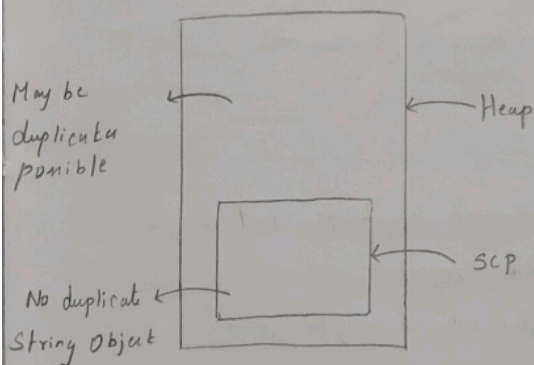
Suddenly, if v2 wants to change Salem → Sathy. Just because of this

Change nearly 1 crore References will be affected and application begins to behave abnormally.

→ Java people analyzed Very clearly, they came up with beautiful Concept "IMMUTABILITY". Once we create String Object, we are not allowed to modify. By mistake if we are trying to change the content, then with those changes a new object will be created.

Why Immutability is Required?

Just because of scp concept only. If Reusing the same object is not possible then Immutability is not Required.



① Why scp concept is available only for String object but not for StringBuffer?

[You - Bawala - duty - One day - no money - privilege - yearly once - no privilege - Analogy]

→ The most commonly used Object in Java is String Object. So, for the String special Memory Management provided by Java ppl.

② Why Strings are Immutable, whereas StringBuffer Object are mutable?

In case of strings, Just because of scp, same objects can be used multiple times. By using one reference, if we are changing the content, the remaining Ref. are going to be affected. That's why Once we created String Object, we are not allowed to modify. This is what people call as "Immutability".

Beoz of scp, strings are Immutable

→ In StringBuffer, scp concept is not there. If the scp concept is not there, then Reusing same object is not there. Every time separate object will be created so no reusing.

"Reusing same object is not there in SB, So Immutability concept not required"

③ In addition to String Object any other Objects are immutable in Java?

Yes, all Wrapper class Objects are Immutable.

String Constructors:

- ① String s = new String();
- ② String s = new String (string);
- ③ String s = new String (StringBuffer);
- ④ String s = new String (StringBuilder);
- ⑤ String s = new String (char[] ch);
- ⑥ String s = new String (byte[] b);

Find the Output:

Class Test

```
psvm (String[] args) {
```

```
String ta = "A";
```

```
ta = ta.concat ("B");
```

```
String tb = "C";
```

```
ta = ta.concat (tb);
```

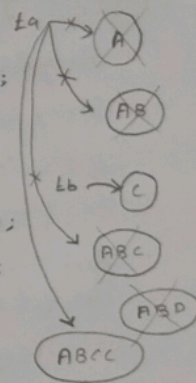
```
ta.replace ('C', 'D');
```

```
ta = ta.concat (tb);
```

```
System.out.println (ta);
```

```
}
```

```
// ABCC
```



Class Test

```
psvm (String[] args) {
```

```
String str = " ";
```

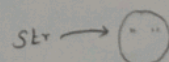
```
str.trim();
```

```
System.out.println (str.equals ("") + "... " +
```

```
str.isEmpty());
```

```
}
```

```
// false ... false
```



Class Tut 1

```

psvm (String[] args) {
    String str1 = "Java";
    String str2 = new String ("java");
    // line -1
    {
        sopln ("Equal");
    }
    else {
        sopln ("Not Equal");
    }
}
}

```

To print "Equal", which code should be placed inside line -1

- A) String s3 = s2;
if (s1 == s3); X
- B) if (s1.equalsIgnoreCase(s2)) { ... } ✓
- C) String s3 = s2;
if (s1.equals(s3)); X
- D) if (s1.toLowerCase() == s2.toLowerCase()) X

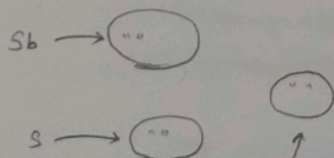
Class Tut 1

```

psvm (String[] args) {
    StringBuilder sb = new SB (5);
    String s = "";
    if (sb.equals(s)) {
        sopln ("match 1");
    }
    else if (sb.toString().equals(s.toString())) {
        sopln ("match 2");
    }
    else {
        sopln ("No match");
    }
}
}

```

// match 2



Equals
as String class equals()
perform content
comparison

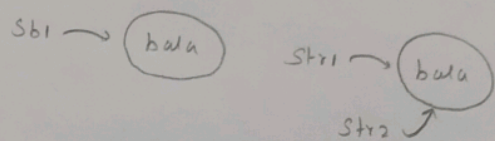
Class Tut 1

```

psvm (String[] args) {
    StringBuilder sb1 = new SB ("bala");
    String str1 = sb1.toString();
    // line -1
    sopln (str1 == str2); // true
}
}

```

- ① String str2 = str1; ✓
- ② String str2 = new String (str1);
- ③ String str2 = sb1.toString();
- ④ String str2 = "Durga"; [created in scp]



Which statement Empty the content of a String Builder Object?

- sb.delete(0, sb.length()) ✓
- sb.deleteAll(); X
- sb.delete(0, sb.size()); X
- sb.removeAll(); X

Class MyString 1

```

String msg;
MyString (String msg) {
    this.msg = msg;
}
public class Tut 1 {
    psvm (String[] args) {
        sopln ("Hello" + new SB ("Hai"));
        sopln ("Hello" + new MyString ("Hai"));
    }
}

```

// HelloHai
HelloMyString @ <hardcode>

You are developing a banking module. You have developed a class named MaskText that has a mask method.

class MaskText {

public static String mask (String creditCard)

{ String x = "XXXX-XXXX-XXXX";

// Line-1

}

public (String[] args) {

System.out.println (mask ("1234-5678-9101-1101"));

}

}

You must ensure that mask method returns a string that hides all digits of the credit card number except last four digits & hyphens that separate each group of 4 digits

A) String Builder sb = new SB (creditCard);

sb.substring (15, 19);

return x + sb; ✗

B) return x + (creditCard.substring (15, 19)); ✓

C) String Builder sb = new SB (creditCard);

String Builder s = sb.insert (0, x);

return s.toString(); ✓

D) String Builder sb = new SB (x);

sb.append (creditCard, 15, 19);

return sb.toString(); ✗

→ Java 9 introduced "compact strings" a major internal optimization. Before Java 9, Every String used a char[] regardless of content. Java 9 introduced "compact string" switching to a byte[] with a code build.

↳ If all characters are Latin-1 (0-255) it uses 1 byte per character [code = LATIN1]

↳ If any character requires full UTF-16, it uses 2 bytes per character [code = UTF16]

↳ Memory utilization improved.

→ Java 11 added repeat (int n) to repeat a string n times. ***

"X".repeat (3) → "XXX"

→ Java 15 stabilized text-blocks (Syntactic-Sugar) for formatting multi-line strings

String html = """

<html>

<body>

<h1> Hello </h1>

</body>

</html>

""";

System.out.println (html);

Output: <html>

<body>

<h1> Hello </h1>

</body>

</html>

java.lang.String ✓

java.lang.StringBuilder ✓

java.lang.StringBuilder ✓

java.util.StringTokenizer

→ The process of dividing a string into number of tokens is called as String Tokenization.

→ To perform String Tokenization JAVA has provided a predefined class in the form of java.util.StringTokenizer.

→ A delimiter is a character that separates tokens (comma, space, semicolon)

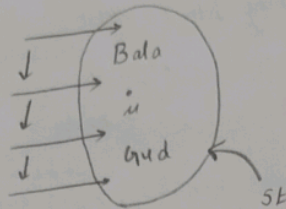
→ The StringTokenizer internally tracks where it is in the string. Every time a token is extracted, the position moves forward.


```

import java.util.*;
class Test {
    psvm (String[] args) {
        StringTokenizer st = new ST ("Bala in Gud");
        sopln (st.countTokens());
        while (st.hasMoreTokens()) {
            sopln (st.nextToken());
        }
    }
}

```

OUTPUT: 3
Bala
in
Gud



- If no delimiter is specified, then the default delimiter set is " \t\n\r".
- These are collectively known as WhiteSpace Characters.
- The behaviour of the StringTokenizer depends on whether returnDelims is true/false.
 - If false: Delimiters only separate tokens and are not included in Output
 - If true: Delimiters consider themselves as valid tokens.

```

class Test {
    psvm (String[] args) {
        String str = "Java, Python, C++ | C#";
        ST st = new ST (str, ",;|");
        while (st.hasMoreTokens()) {
            sopln (st.nextToken());
        }
    }
}

```

OUTPUT: Java
Python
C++
C#

return Delim == false

class Test {

```

    psvm (String[] args) {
        String str = "Java, Python, C++ | C#";
        ST st = new ST (str, ",;|");
        ST st1 = new ST (str, ",;|", true);
        while (st1.hasMoreTokens()) {
            sopln (st1.nextToken());
        }
    }
}

```

OUTPUT: Java
,
Python
;
C++
|
C#

Constructors:

- StringTokenizer (String str)
- StringTokenizer (String str, String delim)
- StringTokenizer (String str, String delim, boolean returnDelim)

Important Methods:

- public boolean hasMoreTokens()
 - True if there are more tokens available from this tokenizer's String
- public String nextToken()
 - Returns the next token
- public int countTokens()
 - Calculates the number of times that this tokenizer's nextToken method can be called before generating an Exception. The current position is not advanced

Class Tut 1

```
psvm (String[] args) {
```

```
String str = "Java, Python, C++ | C#";
```

```
String st = new String (str, ",;|");
```

```
System.out.println (st.countTokens());
```

```
while (st.hasMoreTokens()) {
```

```
    System.out.println (st.nextToken());
```

```
}
```

```
String st1 = new String ("Hello-World:", "-:");
```

```
System.out.println (st1.nextToken());
```

```
System.out.println (st1.nextToken(":-"));
```

```
}
```

```
}
```

OUTPUT: 4

```
Java
Python
C++
C#
Hello
-World
```

→ Although StringTokenizer, is useful, Java developers often prefer split() from String class for better readability.

Class Tut 2

```
psvm (String[] args) {
```

```
String str = "Hello, Java! Hai-You";
```

```
String[] tokens = str.split ("[,!?:]+");
```

```
for (String t : tokens) {
```

```
    System.out.println (t);
```

```
}
```

```
}
```

```
}
```

INTERNING OF STRING OBJECTS:

→ We can use intern() Method to get corresponding SCP Object reference by using heap Object reference. for {

→ By using Heap Object reference, if we want to get corresponding SCP Object reference then we should go for intern() Method.

“The intern() Method helps reduce memory consumption by eliminating duplicate string objects, storing intern strings in JVM heap”

```
String s1 = new String ("bala");
```

```
String s2 = s1;
```

```
System.out.println (s1 == s2); // true
```

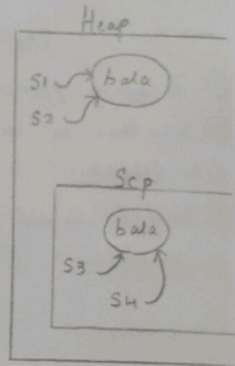
```
String s3 = "bala";
```

```
System.out.println (s1 == s3); // false
```

```
String s4 = s1.intern();
```

```
System.out.println (s3 == s4); // true
```

→ There is no such rule like for every heap object, a corresponding object in SCP must be there. In that case intern() method behaviour?



```
String s1 = new String ("A");
```

```
String s2 = s1.concat ("B");
```

```
String s3 = s2;
```

```
String s4 = s3.intern();
```

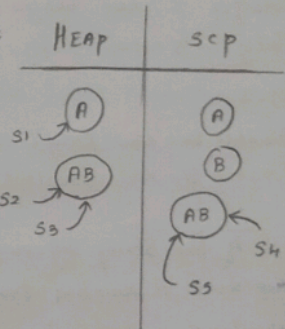
```
System.out.println (s3 == s4);
```

```
System.out.println (s2 == s4);
```

```
String s5 = "AB";
```

```
System.out.println (s4 == s5);
```

<pre>false</pre>	} Before Java 7	<pre>true</pre>	} From Java 7
<pre>false</pre>		<pre>true</pre>	
<pre>true</pre>		<pre>true</pre>	



→ Before Java 7, the scp was in PERMANENT space, separate from heap. Interning a string often creates a distinct copy in the scp, so s4 and s5 might refer a different object than s2 and s3, yielding 'false'.

→ Since Java 7, the scp moved to the heap and interning often reuses the original heap object for efficiency.

↳ If string exists in the scp, it returns that reference. ***

↳ If not, it adds the existing heap object to the scp (rather than copying it) and returns its reference. ***

```
String s0 = "AB";
```

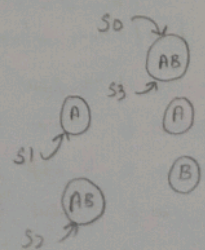
```
String s1 = new String ("A");
```

```
String s2 = s1.concat ("B");
```

```
String s3 = s2.intern();
```

```
System.out.println (s3 == s2); // false
```

```
System.out.println (s3 == s0); // true
```



... check after Autoboxing